AI Digital Memory for Students

Machine Learning-Based Personalized Revision System & Cognitive Retention Engine

Author: Mayuresh (Mayuresh-001) | **Stack:** Python 3.12, FastAPI, Scikit-Learn, SQLite3 | **Scope:** Classes, Dataset, Algorithms, Schema | **Date:** November 2026

1. Executive Summary & Problem Formulation

Students suffer from rapid knowledge decay (the Ebbinghaus forgetting curve) when learning technical material without timely spaced reviews. Static revision timetables fail because learning and forgetting rates vary dynamically by concept difficulty, student test scores, and elapsed time. **AI Digital Memory for Students** maintains an isolated digital memory of every study session and quiz attempt, modeling retention via cognitive spaced repetition and a multi-model Scikit-Learn machine learning pipeline (Logistic Regression, Random Forest, Gradient Boosting) to deliver personalized, diagnostic revision plans.

2. Dataset Architecture & Feature Engineering (ml_dataset • N = 120 Labeled Records)

The system utilizes a cognitive benchmark dataset (ml_dataset) grounded in Hermann Ebbinghaus's spacing effect principles. The dataset tracks 6 continuous/discrete independent variables mapped to a supervised binary retention label:

Feature Name	Data Type	Domain / Range	Cognitive Interpretation & Modeling Role
days_since_last_study	Float	0.5 – 21.0 days	Primary time-decay driver; days elapsed since last active study engagement.
latest_quiz_score	Float	30.0% – 100.0%	Initial encoding strength; higher test score establishes a stronger memory trace.
mistakes_count	Integer	0 – 8 errors	Misconception volume identified during latest knowledge retrieval assessment.
revisions_count	Integer	0 – 5 reviews	Spaced repetition index; completed reviews progressively stabilize retention.
study_duration_minutes	Integer	20 – 90 mins	Session duration indicating depth and cognitive exposure length.
difficulty_level	Integer	1 (Easy) to 3 (Hard)	Inherent complexity weight (Easy=1, Medium=2, Hard=3); hard topics decay faster.
needs_revision (Target)	**Binary**	**0 (Retained) / 1 (Revise)**	**Supervised Target:** 1 indicates memory retention < 55% threshold; 0 indicates retained.

3. Machine Learning Algorithms & Benchmark Evaluation (Held-Out Test Split 25% • 30 Samples)

Features are standardized using StandardScaler (zero mean, unit variance). Models were validated using a held-out stratified test split:

Model Architecture	Algorithm Paradigm	Hyperparameters	Accuracy	Precision	Recall	F1-Score	Status
1. Logistic Regression	Linear Probabilistic Baseline	L2 penalty, C=1.0, max_iter=1000	**90.0%**	90.0%	94.7%	**92.3%**	**Validated**
2. Random Forest	Non-Linear Bagging Ensemble	n_estimators=100, max_depth=5, Gini	**90.0%**	90.0%	94.7%	**92.3%**	**Validated**
3. Gradient Boosting	Sequential Boosting (Log Loss)	n_estimators=80, lr=0.1, max_depth=3	**90.0%**	90.0%	94.7%	**92.3%**	**Validated**

Algorithmic Roles & Ensemble Voting

• **Logistic Regression:** Transparent linear log-odds weights demonstrating positive correlation with elapsed days (+2.42) and mistakes (+1.85).
• **Random Forest:** Models high-order non-linear interactions between difficulty and time decay without overfitting.
• **Consensus Equation:** P_consensus = 0.25*P(LR) + 0.40*P(RF) + 0.35*P(GB).

Gini Feature Importance (Random Forest Classifier)

1. Elapsed Days (days_since_last_study): **38.2%** (primary decay factor)
2. Quiz Mastery (latest_quiz_score): **26.1%** (mastery baseline)
3. Misconceptions (mistakes_count): **17.4%** (confusion penalty)
4. Topic Complexity (difficulty_level): **8.9%** (inherent topic drag)
5. Repetitions Completed (revisions_count): **5.8%** (stabilizer)
6. Study Duration (study_duration_minutes): **3.6%** (exposure volume)

4. Software Architecture & Class Taxonomy

Built in Python 3.12 with FastAPI, Pydantic v2, and SQLite3, separating responsibilities into modular Object-Oriented layers:

Core Python Engine Classes
• **MLEngine** (backend/ml_engine.py): Encapsulates model training, joblib serialization into data/models/, held-out evaluation, and consensus prediction.
• **RevisionPlanner** (backend/revision_planner.py): Hybrid Ebbinghaus retention decay engine and natural-language diagnostic explanation generator.

Pydantic DTO Classes
• **RegisterRequest / LoginRequest:** Authentication DTOs.
• **SubjectCreate / TopicCreate:** Course taxonomy models.
• **SessionCreate / SessionUpdate:** Digital memory session logs.
• **QuizSubmission:** MCQ test answer validation mapping.
• **PredictSandboxRequest:** Real-time ML simulation inputs.

Frontend State & UI
• **state:** Centralized reactive store for tokens, charts, and quizzes.
• **LocalDB:** Client-side localStorage persistence engine.
• **Chart.js:** Canvas controllers for score trends, weekly study bar charts, and revision priority doughnut charts.

5. Relational Database Schema & Entities (SQLite3 • Foreign Key Cascade Integrity)

Table Name	Primary / Foreign Keys	Key Attributes & Constraints	System Function & Data Role
students	id (PK)	name, email (UNIQUE), password_hash, salt, goals	PBKDF2-HMAC-SHA256 authenticated student accounts (100k rounds).
sessions	token (PK), student_id (FK)	created_at, expires_at (7-day validity)	Active bearer token sessions with automatic expiration validation.
subjects	id (PK), student_id (FK)	name, code, color (#hex), description	Academic courses (e.g., Machine Learning, Data Structures, Linear Algebra).
topics	id (PK), subject_id, student_id	title, difficulty ('Easy','Medium','Hard'), notes	Granular syllabus modules associated with parent subjects.
study_sessions	id (PK), topic_id, student_id	study_date, duration_minutes, quiz_score, mistakes	**Digital Memory:** Persistent chronological record of every study event.
quiz_questions	id (PK), topic_id (FK)	question_text, option_a..d, correct_opt, explanation	Curated multiple-choice questions for knowledge testing and retrieval practice.
quiz_attempts	id (PK), topic_id, student_id	score (%), correct_count, mistakes_count, taken_at	Assessment outcomes feeding retention models and mistake counters.
revisions	id (PK), topic_id, student_id	revision_date, notes, outcome ('completed')	Spaced repetition intervention log recording concept reinforcement.
ml_dataset	id (PK)	days, score, mistakes, revisions, duration, diff, target	Supervised benchmark training dataset for scikit-learn model fitting.

6. Hybrid Recommendation Engine & Mathematical Formulation

A. Memory Stability Equation (S) & Ebbinghaus Retention (R_rule):
$S = 2.5 * (1.0 + 0.7 * N_{rev}) * \max(0.3, Score / 100) * M_{diff} * \max(0.5, 1.0 - 0.12 * Mistakes)$
where M_{diff} in {Easy: 1.35, Medium: 1.00, Hard: 0.75} | $R_{rule} = \exp(-\Delta t / S) * 100\%$
B. Hybrid Retention Score (R_hybrid) & Priority Classification Tiers:
$R_{hybrid} = 0.50 * R_{rule} + 0.50 * (1.0 - P_{consensus}) * 100\%$
• **HIGH PRIORITY:** $R_{hybrid} < 45\%$ OR $\Delta t \geq 7$ days OR Score < 60% --> *Revise Today*
• **MEDIUM PRIORITY:** $45\% \leq R_{hybrid} < 65\%$ OR $\Delta t \geq 4$ days --> *Schedule Review Soon*
• **LOW PRIORITY:** $R_{hybrid} \geq 65\%$ --> *Concept Well-Retained*

7. Production Verification & Access Endpoints

Automated Verification (test_app.py)
Passed 10/10 automated tests: PBKDF2 authentication, isolated student queries, session logging, quiz auto-grading, dynamic Ebbinghaus decay, ML inference, and dashboard KPIs.

Live Access & Deployment Endpoints
• **Live Web Application:** https://mayuresh-001.github.io/AI-digital-memory-/
• **GitHub Repository:** https://github.com/Mayuresh-001/AI-digital-memory-
• **Local Server:** http://127.0.0.1:8000 (FastAPI / Uvicorn runner)